

D3.4

Co-Design and Energy Efficiency

Report II

Project Title	dealii-X: an Exascale Framework for Digital Twins of the Human Body
Project Number	101172493
Funding Program	European High-Performance Computing Joint Undertaking
Project start date	1 October 2024
Duration	27 months



dealii-X has received funding from the European High-Performance Computing Joint Undertaking Programme under grant agreement N° 101172493

Deliverable title	Co-Design and Energy Efficiency Report II
Deliverable number	D3.4
Deliverable version	v1.0
Date of delivery	July 31, 2026
Actual date of delivery	July 31, 2026
Nature of deliverable	Report
Dissemination level	Public
Work Package	WP3
Partner responsible	BADW-LRZ

Abstract	This report focuses on development and porting of finite element kernels to GPU architectures and analysis of energy efficiency.
Keywords	codesign; performance; energy

Document Control Information

Version	Date	Author	Changes Made
0.1	29/07/2026	Ivan Pribec	Initial Draft
0.2	30/07/2026	Enes Soydan	Added Performance Results
0.3	31/07/2026	Gerald Mathias	Review
1.0	31/07/2026	Martin Kronbichler	Final version

Approval Details

Approved by: Martin Kronbichler

Approval Date: 31/07/2026

Distribution List

- Project Coordinators (PCs)
- Work Package Leaders (WPLs)
- Steering Committee (SC)
- European Commission (EC)

Disclaimer: This project has received funding from the European Union. The views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the European High-Performance Computing Joint Undertaking (the “granting authority”). Neither the European Union nor the granting authority can be held responsible for them.

Contents

1	Introduction	4
1.1	Purpose of the Document	5
1.2	Relation to Other Deliverables	5
1.3	Structure of the Document	6
2	Co-design for High-Performance Finite Element Kernels	7
2.1	Batched Operator Evaluation	7
2.2	Experimental Setup	8
2.3	Operator Performance and Optimization Analysis	8
2.4	Laplace Solver Performance (CEED BP3)	10
2.5	Raviart–Thomas Elements and Kernel Performance	12
2.6	Alternative Programming Models	13
2.6.1	Tiny Tensor Compiler: Status Update	13
2.6.2	Kernel Specialization and Compilation Trade-offs	14
3	Energy Efficiency of High Order Finite Element Kernels	17
3.1	Metrics	18
3.2	Measurement methodology	18
3.3	The efficiency–performance plane	21
3.4	Preliminary results	21
3.5	Work plan	22
4	Conclusion	24

1 Introduction

This is the second co-design and energy-efficiency report of the dealii-X project. It is a direct continuation of Deliverable D3.2, *Co-Design and Energy Efficiency Report I*. The motivation for the work, the selection of the CEED Bake-off Kernels (BK) and Bake-off Problems (BP) [Fischer et al., 2020] and the streaming kernels [Chalmers and Warburton, 2020] as proxies for the algorithmic core of high-order finite element codes, and the description of the benchmark repository¹ are given in D3.2 and are not repeated here.

D3.2 established a performance baseline for the scalar operator-evaluation kernels BK1, BK3 and BK5 on NVIDIA, AMD and Intel accelerators, using a range of programming models (CUDA, Kokkos, OpenCL, OpenMP target offloading and the Tiny Tensor Compiler), together with an initial characterization of the streaming kernels on CPUs. It closed by identifying three directions for the remaining project period: the exploitation of matrix instructions and cross-element vectorization, the extension from isolated operator evaluation to the complete solver chain including multigrid preconditioners, and the adoption of a portable, process-based methodology for energy measurement. Here we report on the outcome of those three directions.

Since D3.2, the performance optimization effort has concentrated on three new areas:

- **Batched (thread-coarsened) kernels.** The single-element-per-thread-block mapping used in D3.2 leaves registers, shared memory and – on AMD hardware – the wavefront underutilized at low polynomial order. Assigning several elements to each thread block removes this limitation and yields a $2\times$ speedup for $p = 1$ on the NVIDIA GH200 (Section 2). This directly addresses the poor low-order performance that was observed but left unresolved in D3.2.
- **Raviart–Thomas vector element kernels.** The kernels studied in D3.2 were exclusively H^1 -conforming and scalar-valued elements. Mixed formulations of the Navier–Stokes equations highly benefit from $H(\text{div})$ -conformity in terms of robustness, and we therefore extend the benchmark suite with the vector mass, mixed divergence and mixed gradient operators of the Raviart–Thomas element family.

¹<https://github.com/dealii-X/benchmarks/>

- **Full Bake-off Problems including solvers and preconditioners.** Operator evaluation is embedded in a conjugate gradient solver and in multigrid-preconditioned solvers within the deal.II framework, so that gather–scatter overheads, inter-GPU communication and preconditioner cost enter the measurement. This closes the gap between the BK and BP figures of merit that was left open in D3.2.

Where D3.2 could only outline the process-based measurement strategy of [Cielo et al. \[2025\]](#), this report presents measurements obtained with the `p3em` energy meter and an explanation of the energy efficiency metrics.

1.1 Purpose of the Document

The purpose of this document is to report the co-design outcomes of Work Package 3 in the second half of the project: which implementation techniques were found to reach hardware limits on current accelerators, which programming models are mature enough to be relied upon, and what the associated energy cost is. The findings are intended to inform the library developments in WP1 and the application developments in WP2, as well as the wider finite element and GPU computing communities.

The performance-portability objectives stated in D3.2 remain unchanged: programs should deliver good performance across diverse computing architectures while requiring only minor platform- and simulation-specific tuning.

1.2 Relation to Other Deliverables

To avoid duplication, the following material is documented elsewhere and only referenced here:

- **D1.3** – the matrix-free framework and its initial deal.II implementations.
- **D1.7** – extensive BP results on NVIDIA, AMD and Intel hardware, and the portable GPU multigrid preconditioners.

- **D3.2** – motivation, benchmark definitions, the programming-model survey and the baseline BK1/BK3/BK5 and streaming-kernel results.
- **D3.3** – the performance-analysis tool chain (NVIDIA Nsight Systems, Intel VTune and Intel Advisor, LIKWID) and the description of the `p3em` energy-measurement library and its instrumentation API.

1.3 Structure of the Document

- Section 2: Co-design for High-Performance Finite Element Kernels – batched operator evaluation, Raviart–Thomas kernels, solver-level (BP) performance, and an updated assessment of alternative programming models.
- Section 3: Energy Efficiency – metrics, measurement methodology and results.
- Section 4: Conclusions and Outlook.

2 Co-design for High-Performance Finite Element Kernels

We detail the implementation techniques and performance measurements of high-order matrix-free finite element methods on contemporary graphics processing units (GPUs). The matrix-free framework and its initial implementations are described in Deliverable 1.3 (https://github.com/dealii-X/deliverables/blob/main/D1.3/D1.3_dealii_report.pdf). To evaluate these problems, we adopt the bake-off benchmarks Fischer et al. [2020] as our baseline problems. In this report, we also implement a new technique to improve operator performance for low-order elements.

2.1 Batched Operator Evaluation

Previous high-order matrix-free GPU implementations were based on assigning a single element per thread block [Świrydowicz et al., 2019]. This approach causes low-order elements to underutilize the streaming multiprocessors (SMs) because NVIDIA only allows each SM to schedule 32 thread blocks [NVIDIA Corporation, 2026a]. Consequently, low-order elements reach this limit, leaving registers and shared memory underutilized within each SM. On AMD GPUs, the single element per thread block approach leaves the 64-thread wavefront underutilized for low-order elements, consequently causing poor performance. Based on a batch processing approach, our technique assigns multiple elements to each thread block to overcome these limitations. For example, a polynomial order of 1 in the Laplace operator evaluation (Bake-off Kernel 3 in the CEED benchmarks) can fit 11 elements per thread block. A performance comparison between the single-element-per-thread-block and the batched processing approaches is given in Figure 1. The new technique achieves a $2\times$ speedup for polynomial order 1 on the NVIDIA GH200 Superchip at the JUPITER supercomputer located at the Jülich Research Centre. New batched implementations are available at https://github.com/dealii-X/benchmarks/tree/main/CEED_BK.

2.2 Experimental Setup

Performance analysis and benchmarks were conducted on the NVIDIA Grace Hopper Superchip (GH200) on the JUPITER supercomputer and on the AMD MI250X GPU on the LUMI supercomputer. The Kokkos performance portability layer v5.1 [Trott et al., 2022] was used to target traditional compute cores. For tensor core implementations, CUDA PTX mma instructions v9.3 [NVIDIA Corporation, 2026b] were employed.

2.3 Operator Performance and Optimization Analysis

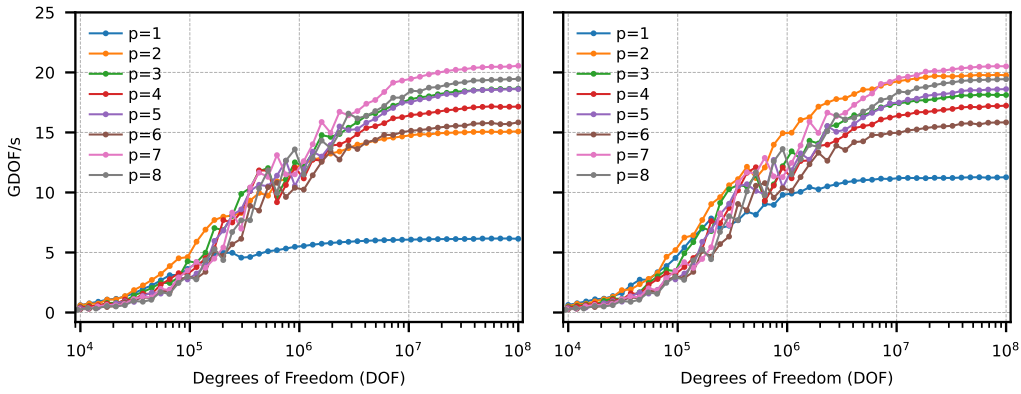


Figure 1: Performance comparison of the Laplace operator (BK3) in FP64 on the NVIDIA GH200. (Left) Single-element-per-thread-block approach; (Right) Proposed multiple-elements-per-thread-block approach .

To see the occupancy impact of our proposed approach, we investigate the performance of 12.5 million linear elements ($p = 1$) in double precision using the NVIDIA Nsight Compute profiler. Figure 2 shows a screenshot of the occupancy calculator for the single element per thread block kernel. Each box represents a thread block, where light blue boxes represent allocated blocks and dark blue boxes represent unallocated blocks. We can clearly see that 18.8% of registers and 31% of shared memory are left unused due to the 32-thread-block-per-SM limit. On the other hand, Figure 3 shows 10 allocated blocks. In this case, higher register usage becomes the occupancy limiter, resulting in improved performance.

To further analyze performance, we also leveraged NVIDIA Nsight Compute's roofline model to effectively compare a kernel's achieved performance with the hardware bandwidth and FLOP/s limits. Figure 4 shows the hierarchical roofline model

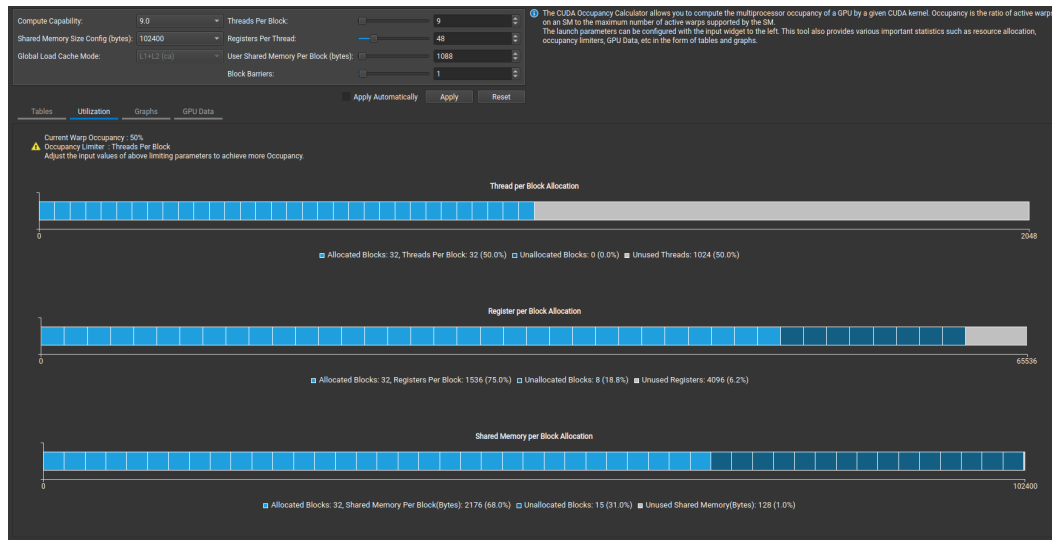


Figure 2: Occupancy calculator screenshot from NVIDIA Nsight Compute for the single element per thread block technique on laplace kernel, evaluated using 12.5 million linear elements in double precision.

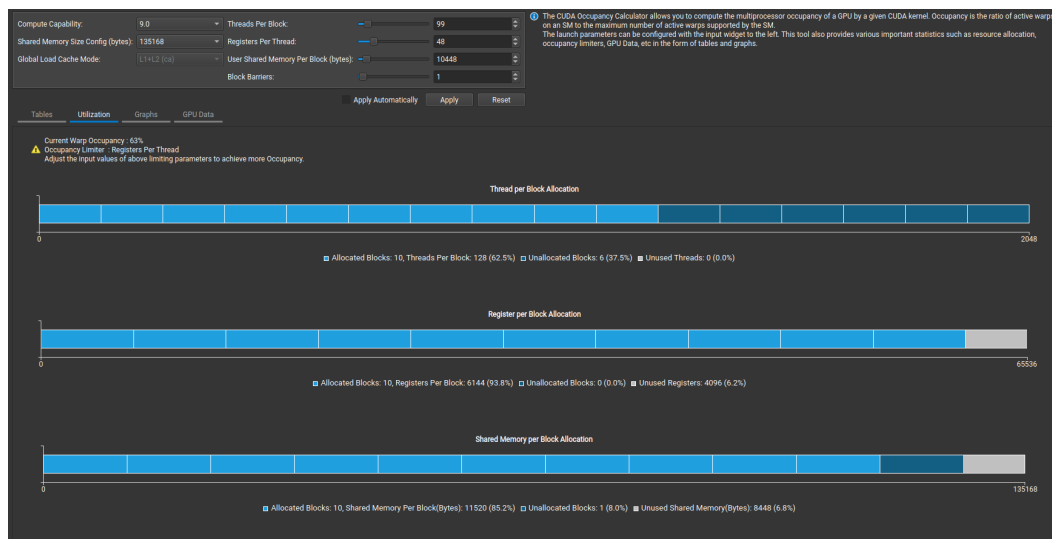


Figure 3: Occupancy calculator screenshot from NVIDIA Nsight Compute for the batched processing technique on laplace kernel, evaluated using 12.5 million linear elements in double precision.

for the single element per thread block technique, evaluated with the same 12.5 million linear element Laplace kernel. The plot includes the L1, L2 and global memory bandwidth boundaries. On this target GH200 GPU, the streaming global memory bandwidth is 3.6 TB/s, whereas its theoretical FP64 peak performance is 33 TFLOPS. Due to the low arithmetic intensity of low-order elements in matrix-free methods, polynomial order 1 has a global memory arithmetic intensity of only 1.3 FLOP/B, achieving a performance of 1.4 TFLOPS. This achieves only 27% of the theoretical roofline limit (5.2 TFLOPS) at that arithmetic intensity. In contrast, the batched processing approach achieves 2.5 TFLOPS at the same arithmetic intensity, reaching 48% of the theoretical throughput limit.

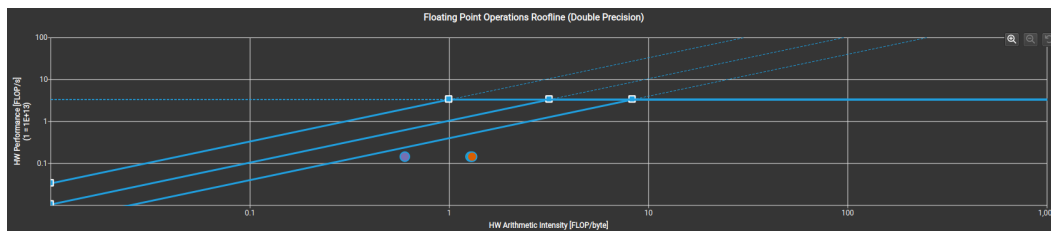


Figure 4: Roofline model from NVIDIA Nsight Compute for the single element per thread block laplace kernel, evaluated using 12.5 million linear elements in double precision. The kernel achieves an arithmetic intensity of 1.3 FLOP/B in global memory and a performance of 1.4 TFLOPS (green solid circle), corresponding to 27% of the theoretical roofline limit (5.2 TFLOPS) at this arithmetic intensity.

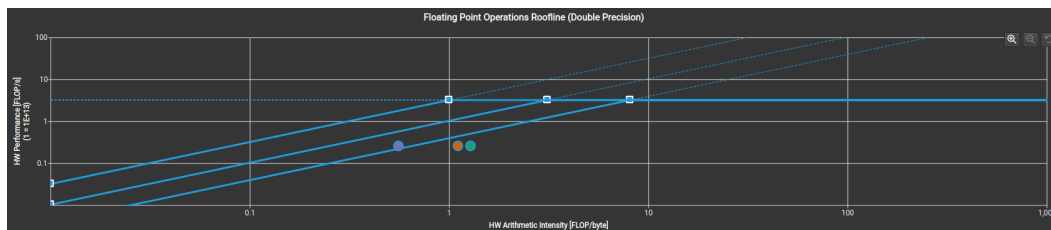


Figure 5: Roofline model from NVIDIA Nsight Compute for the batch processing laplace kernel, evaluated using 12.5 million linear elements in double precision. The kernel achieves an arithmetic intensity of 1.3 FLOP/B in global memory and a performance of 2.5 TFLOPS (green solid circle), corresponding to 48% of the theoretical roofline limit (5.2 TFLOPS) at this arithmetic intensity.

2.4 Laplace Solver Performance (CEED BP3)

We extend our operator evaluation to a complete linear solver by embedding the Laplace operator within a Conjugate Gradient (CG) method, corresponding to the CEED Bake-off Problem 3 (BP3) benchmark. The solver, implemented within the

deal.II finite element library framework is evaluated on both single and multi-GPU configurations of the NVIDIA GH200 Superchip, utilizing MPI with peer-to-peer (P2P) communication enabled. Figure 6 illustrates the initial performance results, highlighting that the performance difference between operator evaluations (BK) and full linear solvers (BP) is primarily driven by gather-scatter overheads and inter-GPU communication. Additionally, the results confirm the computational efficiency of high-order elements which directly leverage the tensor-product formulation of the shape functions to increase arithmetic intensity. The source code can be found at https://github.com/dealii-X/benchmarks/tree/main/CEED_bp.

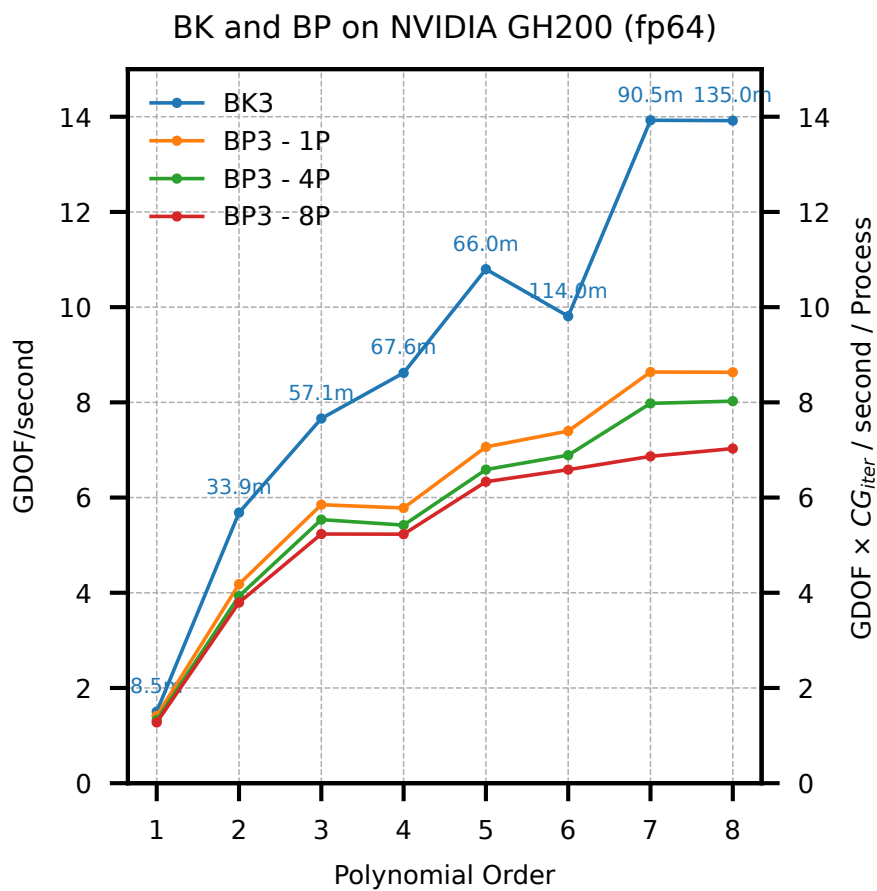


Figure 6: Performance results for the Laplace Operator (CEED BK3) & Problem (CEED BP3) are evaluated across various polynomial orders and problem sizes on the GH200 GPU. Both Bake-off Kernel performance (GDoF/s) and Bake-off Problem performance ($\text{GDoF} \times \text{CG}_{\text{iter}} / (\text{second} \times \text{process})$) are measured using the global degrees of freedom (the T-vector in CEED).

2.5 Raviart–Thomas Elements and Kernel Performance

H^1 -conforming elements (as used in the CEED Bake-off kernels) enforce continuity of all vector components across elements, whereas the Navier-Stokes equations, in mixed formulations often require continuity of the normal component of the flux across element boundaries which is a property of $H(\text{div})$ -conforming spaces. One element type designed for this purpose are the Raviart–Thomas finite elements. We implement the vector mass, mixed divergence and mixed gradient operators – the core operations in Raviart–Thomas kernel design – in our benchmarks repository: https://github.com/dealii-X/benchmarks/tree/main/rt_kernels.

Figure 7 shows the performance of the vector Mass operator on the NVIDIA GH200 in both FP32 and FP64 precisions. Batch processing enabled all polynomial orders to achieve similar throughput (GDoF/s). Similar trends are observed for the AMD MI250X GPU from the LUMI supercomputer, which performs well given its 1.3 TB/s STREAM bandwidth. Only polynomial order 8 performs significantly worse than the other orders, as shown in Figure 8. This drop in performance is likely due to register file capacity constraints in this slightly older AMD architecture.

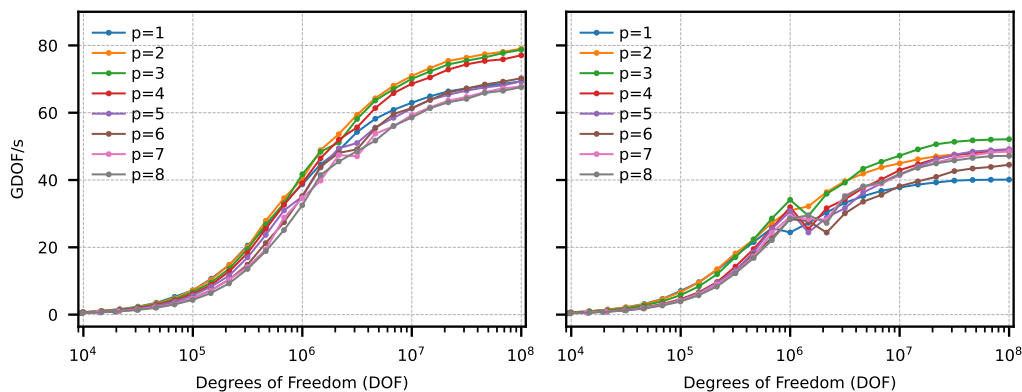


Figure 7: Performance comparison of the vector Mass operator using FP32 (left) and FP64 (right) precisions on the NVIDIA GH200 Superchip.

We also introduce a new technique to target NVIDIA tensor cores [NVIDIA Corporation, 2026b] which are designed to perform fast matrix-matrix multiplication in different precisions. Initial implementations for FP64 and non-IEEE-compliant TensorFloat-32 (TF32) are available in the same `rt_kernels` benchmarks repository folder.

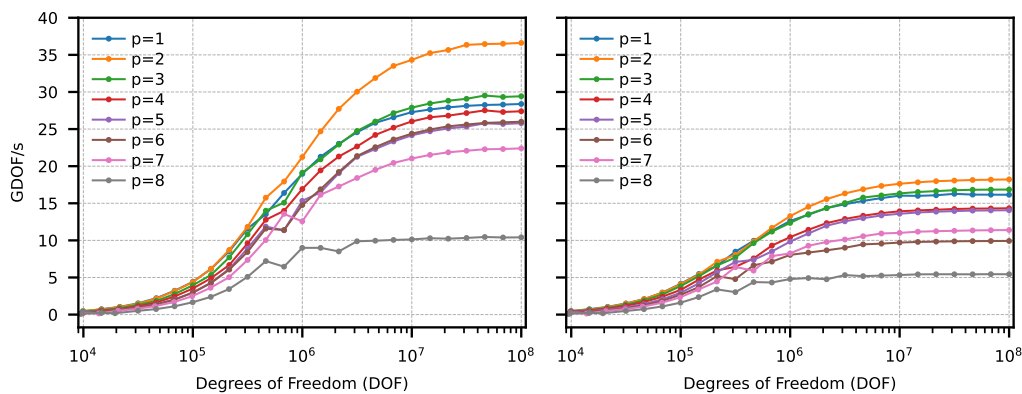


Figure 8: Performance comparison of the vector Mass operator using FP32 (left) and FP64 (right) precisions on the single AMD MI250X GPU tile.

2.6 Alternative Programming Models

The survey of GPU programming models begun in D3.2 has continued in support of the Work Package 1 developments, the aim being that the library implementations exploit accelerators from all three major vendors. The main effort in this period went into optimizing kernels on Intel hardware with the Tiny Tensor Compiler, left as an open problem in the first report.

The model comparison of D3.2 has by now settled into a consistent picture. OpenMP target offloading performs adequately at low polynomial orders but struggles with the arithmetically intensive kernels, offering too little control over shared memory allocation and register pressure. OpenCL matches Kokkos; its drawback is uneven vendor support, NVIDIA profiling tools in particular lacking native OpenCL support, which complicates the analysis reported in Section 2. High-level abstractions thus remain convenient for simpler workloads but lack the granularity to approach machine limits in high-order finite element computations, while among the lower-level portable models the deciding factor is less raw performance than the maturity of the surrounding tooling.

2.6.1 Tiny Tensor Compiler: Status Update

TinyTC, its tensor language and its execution model are described in D3.2, Section 2.4, with a complete Tensor IR listing of the prototype BK1 kernel. There, the TinyTC implementation of BK1 reached about 10 GDoF/s on the Intel Data Center

GPU Max (Ponte Vecchio) – a quarter of the OpenMP throughput – and dimension fusion alone gained only a few percent. In consultation with the compiler author we concluded that index fusion, batch vectorization and memory layout transformations would be needed for satisfactory register and SIMD utilization on that architecture.

That strategy has since been implemented and the kernel set extended from BK1 to BK5. Both kernels now run close to the achievable machine bandwidth of the target Intel GPUs. Performance is sustained across a broad range of polynomial orders, unlike the OpenMP implementations noted above; the residual loss at very high orders is a tuning rather than a structural problem, attributable to shared local memory allocation and padding.

TinyTC exemplifies a wider trend. Tile-based domain-specific languages such as Triton and NVIDIA’s cuTile are attracting substantial commercial investment through their role in neural network training. The common principle is to delegate fine-grained parallelism – thread mapping, vectorization, specialized matrix units – to the compiler, leaving the developer free to concentrate on operation fusion and software pipelining. Matrix-free finite element kernels, whose inner structure is a sequence of small tensor contractions, fit this division of labour naturally, which is why we consider such toolchains relevant beyond Intel hardware.

2.6.2 Kernel Specialization and Compilation Trade-offs

These kernels consist of nested loops with small trip counts, so compile-time loop bounds give the compiler backend considerably more to work with. D3.2 reported a throughput gain of at least 30% for the templated BK1 and BK5 kernels. That figure is a lower bound rather than a characteristic value: the gap to the achievable peak of the device proved larger than the comparison against the untemplated baseline suggested. For reference, [Kronbichler and Kormann \[2019\]](#) report roughly a factor of three from the same optimization on CPUs, attributing it to complete unrolling of the short inner loops and a better instruction schedule, and likewise find the effect most pronounced at low polynomial degree. The kernels of this report gain at least as much as those of D3.2.

Two mechanisms are at work, on different architectures. On Intel GPUs the expected one: with static trip counts the compiler unrolls the loop nest fully and fuses loops into favourable execution patterns. This is pronounced at low polynomial order

but does not carry over to high orders, where the unrolled loop body outgrows what the compiler handles well. On NVIDIA GPUs, where the kernels are thread-based, a second and more persistent mechanism dominates. Recovering the tensor-product indices (i, j, k) of a degree of freedom from its linear index requires integer division and modulo, among the more expensive scalar instructions on these devices. Templating the quadrature point counts makes the divisors compile-time constants, letting the compiler substitute a multiply-and-shift sequence. Analysis with Nvidia Nsight Compute bears this out: for a variant with two-dimensional thread mapping within the block, kernels with runtime loop bounds issue some 8.8 times as many integer instructions as their statically sized counterparts.

Static specialization therefore pays off for different reasons on different targets; on thread-based kernels the benefit is tied to index arithmetic rather than to a favourable unrolling decision.

Its price is compilation time and library size, both growing with the number of instantiated variants. So far this has remained acceptable, and the implementations reported here use static templating throughout. Should the trade-off worsen, several mechanisms retain compile-time constants without instantiating the full variant space ahead of time:

- **CUDA Jitify**, for runtime kernel compilation and caching on NVIDIA hardware.
- **SYCL specialization constants**, for injecting configuration-dependent constants at runtime within oneAPI.
- **TinyTC**, JIT-based by design, generating code specialized to the tensor shapes encountered at execution time – anticipated in D3.2 as a prospective advantage of a tensor DSL.

Others in the community pursue the same goal through domain-specific languages and finite-element-tailored intermediate representations. For example, libCEED implements a just-in-time compiler as well. A practical constraint here is that Kokkos, on which the portable deal.II kernels are based, offers no such abstraction; for Kokkos-based application codes it may therefore be preferable to fix the polynomial order at build time rather than dispatch on it at runtime.

Which route best suits a general-purpose finite element library remains open, and the answer will likely depend on maintainability and build-system considerations

as much as on the performance figures reported here.

3 Energy Efficiency of High Order Finite Element Kernels

This section describes ongoing work in task WP3.4 on energy and performance profiling. The object of study is the matrix-free evaluation of high-order finite element operators, i.e., the kernels that dominate the run time of implicit and explicit high-order solvers. These kernels are characterised by a small arithmetic footprint per degree of freedom at low polynomial degree and a rapidly growing one at high degree, so that the same operator traverses the memory-bound and the compute-bound regime as the degree is varied. This makes the family a convenient probe of the energy behaviour of a node, and it also places the code-generation strategy and the compiler among the parameters under study rather than outside them.

The motivation is one of scale. A single NVIDIA H100 held at 61 % of its 700 W thermal design power over a full year consumes

$$0.700 \text{ kW} \times 0.61 \times 8760 \text{ h} \approx 3740 \text{ kWh},$$

which is more than the 3500 kWh of the reference household commonly used in German electricity statistics, and this is the bare accelerator, before cooling, the host and the rest of the node are accounted for. A production system aggregates tens of thousands of such devices: JUPITER drew close to 16 MW during its exascale TOP500 run, with an average operational demand estimated at some 11 MW, which over a year is of the order of the household electricity consumption of a small town. At this scale the Joules spent by a kernel are a first-order concern, and energy-conscious HPC — treating energy-to-solution as a figure of merit on the same footing as time-to-solution — becomes a design constraint on codes and not only on machines. This is the perspective from which the present task approaches EuroHPC systems.

The massive parallelism and comparatively low clock frequency of GPUs can offer significant advantages for scientific codes in comparison to CPUs. Power, however, grows super-linearly with frequency, so the operating point that minimises time-to-solution is in general *not* the one that minimises energy-to-solution. Quantifying the gap between the two, for a kernel class of practical relevance, is the purpose of this task.

3.1 Metrics

Energy-to-solution is, by definition, the integral of the instantaneous power dissipation over the measured interval,

$$E = \int_0^T P(t) dt. \quad (1)$$

The power dissipation P can in principle be modelled as a function of clock frequency and of the achieved performance, and there is a substantial literature doing so. Within WP3.4 we measure E directly and proceed empirically, with the aim of establishing where our kernels currently stand and what can be gained by tuning them, rather than of producing a predictive power model.

Performance is reported as the number of degree-of-freedom updates per second (GDoF/s), the customary throughput metric for matrix-free operator evaluation. Energy efficiency is reported as the number of *DoF updates per Joule*, which is simply the throughput normalised by the average power drawn during the measured region,

$$\eta = \frac{\text{DoF updates}}{E} = \frac{\text{DoF updates}/t}{\bar{P}}. \quad (2)$$

Reporting η alongside GDoF/s makes the trade-off explicit: a configuration that is faster but disproportionately more power-hungry shows up immediately as a loss in η .

Where a single figure of merit is needed, the energy–delay product $\text{EDP} = E \cdot t$ serves as a secondary metric. It penalises an operating point that lowers energy merely by running longer, which a criterion counting Joules alone would reward. We report η and R separately throughout.

3.2 Measurement methodology

Measurements are driven by `p3em` [Cielo et al., 2025], an open source approach to power, performance and energy measurement [Pribec et al., 2026]. Rather than wrapping the benchmark binary as a whole, it instruments a local scope of the program: a background thread queries a power meter running as a separate process and integrates the returned power readings over the lifetime of the scope. Three properties of this design matter for the study. First, it is selective: the measured

region can be restricted to the parts of the program that are of interest, such as the operator evaluation, the preconditioner or the solve, while excluding I/O and set-up phases that would otherwise dilute the result. Second, it is portable, since support for a new machine requires only a different meter script, the rest of the instrumentation being unchanged; in the common case the whole chain runs in user space, without privileged access. Third, we can measure CPU and GPU energy usage separately, accounting for different phases of the process.

The approach is not specific to the kernels studied here. It has been applied to a set of flagship science applications on SuperMUC-NG Phase 2, in a study presented at the IXPUG workshop of ISC 2026 which documents the methodology and its validation in more detail [Cielo et al., 2026]. That work measures whole applications rather than isolated kernels, and is the natural counterpart to the kernel- and problem-level measurements reported below.

Around this instrumentation, the study of a given system consists of the following components:

- Investigating the operating points exposed by the device and by the host: the clock frequencies the hardware supports and the range over which they may be set, together with any power-cap and memory clock settings that are available.
- Setting an operating point and running the kernel sweep (polynomial degree, number of elements, precision) with the instrumented region active.
- Recording one entry per (kernel, degree, operating point) tuple, so that performance and energy come from the same run and are directly comparable.

The meter scripts build on NVML on NVIDIA devices, the AMD SMI library on AMD devices, and the Level Zero Sysman interface on Intel devices. Three methodological points require care and are documented together with the results: the sampling rate of the meter and its latency relative to short measured regions, the treatment of idle power, and the attribution of energy on tightly coupled packages such as the GH200 superchip, where the CPU, the GPU and the memory share a configurable power budget.

Systematically analysing power limits and frequency settings in this way allows the project to make informed decisions about optimising energy efficiency while

maintaining a satisfactory level of performance, and to state the cost of that trade-off in concrete terms, e.g. the energy saved at a prescribed loss in time-to-solution.

Tables 1 and 2 collect the devices of interest. Where possible the data is taken from the models used in the EuroHPC JU systems. A model name does not identify a single product, however: form factor, memory configuration, enabled SM count and clock range all vary between variants, so the figures describe the specific parts installed in the named systems rather than the model as such.

Table 1: Characteristics of the Nvidia and Intel GPUs of interest. The figures describe the parts installed in the named systems, not the catalogue specifications of a model.

	Nvidia A100	Nvidia H100	Nvidia GH200	Intel GPU Max 1550
Variant	SXM 64GB	64GB	superchip	–
System	Leonardo	MareNostrum5	JUPITER	SuperMUC-NG Ph. 2
TDP [W]	440	350-400	680 (superchip)	600
FP32 [TFLOPS]	22.4	60	67	52.4
Peak Bandwidth [TB/s]	1.64	3.9	4	3.2
Base Frequency [MHz]	1245	–	–	900
Boost Frequency [MHz]	1395	–	1980	1600
Memory Size [GB]	64	64	96	128
Memory Type	HBM2	HBM2e	HBM3	HBM2e

Table 2: Characteristics of AMD GPUs of interest. The MI430X has not been released at the time of writing; its figures are pre-production estimates.

	AMD MI250X	AMD MI300X	AMD MI430X
System	LUMI	–	Alice Recoque
TDP [W]	500	750	2500 (est.)
FP32 [TFLOPS]	47.9	81.72	288 (fp64)
Peak Bandwidth [TB/s]	3.28	5.32	19.6
Base Frequency [MHz]	1000	1000	–
Boost Frequency [MHz]	1700	2100	–
Memory Size [GB]	128	192	432
Memory Type	HBM2e	HBM3	HBM4

3.3 The efficiency–performance plane

The most informative single view of a frequency sweep plots energy efficiency η (GDoF/J) on the vertical axis against performance R (GDoF/s) on the horizontal axis, with the clock frequency or the power cap as the parameter along the curve. Both quantities are oriented so that larger is better, so the desirable region is the upper right corner. This way of presenting the trade-off follows the analysis of power capping and frequency scaling given by [Hager et al. \[2026\]](#). How the curves are shaped depends heavily on whether the kernel is memory bound or compute bound, and beyond that there is little we can say before the measurements have been made. Since the polynomial degree moves our kernels between these two regimes, the family of curves should change character across the degree range.

The same plane may accommodate more than the frequency knob: curves parameterised by polynomial degree, by the choice of algorithm or implementation, or by compiler and code-generation strategy can be drawn on the same axes. It thereby shows in one picture which combination of runtime and build-time parameters places a given application where it wants to be on the trade-off.

Better algorithms, better methods and better implementations can move energy-to-solution by a larger factor than any frequency or power cap. Mixed precision is the obvious example: evaluating the operator in single precision, or storing geometry and interpolation data in reduced precision while accumulating in double, cuts the data volume moved and thereby the dominant energy cost for the memory-bound part of the degree range. Such choices are not free, however, and have to be weighed carefully against the accuracy the application requires. The distinction matters for how the two are treated: a power cap can be changed and reverted without affecting the computed result, whereas a change of precision changes the answer and must be justified numerically. The two lines of work are complementary, and the plane described above is precisely the instrument for placing them on common axes.

3.4 Preliminary results

For the BK1 mass operator we have measured the performance and the energy efficiency of the OpenMP, Kokkos and TinyTC implementations on a single device at

its default operating point, shown in Figure 9 and presented in preliminary form at the WCCM-ECCOMAS 2026 congress by [Pribec et al. \[2026\]](#). The two groups of bars are essentially proportional to one another: the ranking of the implementations by GDoF/s is also their ranking by DoF updates per Joule, and the ratios between them are nearly the same in both metrics. This is what one expects whenever the average power drawn is similar for all variants, since $\eta = R/\bar{P}$ then reduces to R up to a common factor; the recorded power traces allow this reading to be checked directly, and $\bar{P}$ is therefore reported alongside the two metrics. The practical consequence is a negative but useful result. At a fixed operating point there is no trade-off to be made, and an implementation optimised for time-to-solution is also the one optimised for energy-to-solution. A trade-off can only appear once the operating point itself is varied. No frequency or power-cap sweep has been performed so far, and this is precisely the motivation for the study described below.

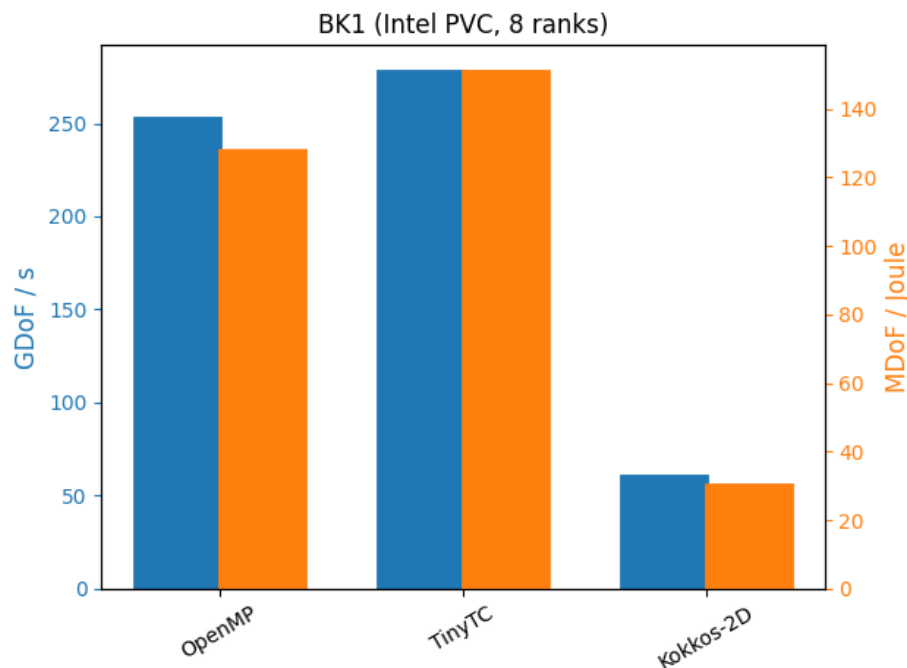


Figure 9: Performance (GDoF/s) and energy efficiency (DoF updates per Joule) of the OpenMP, Kokkos and TinyTC implementations of the BK1 mass operator for degree $p = 2$ in 32-bit precision. The setup consists of 1000 billion elements per GPU tile (Intel Ponte Vecchio).

3.5 Work plan

The work planned for months M17–M22 of the project comprises:

- Consolidation of `p3em`: a uniform output format across meter scripts, documented sampling and idle-power conventions, and documentation of which part of the package each device counter actually measures, so that figures obtained on different vendors' hardware can be brought onto a common basis.
- Extension of coverage on a single reference GPU, at both kernel and problem level, over the polynomial degree range of interest and in single, double and mixed precision, for the implementations already compared above as well as our own CUDA/Kokkos variants.
- A cross-platform campaign at default settings on the devices of Tables 1 and 2, with identical benchmarks, metrics and sweep configuration.
- The frequency and power-cap scaling study: η and R measured across the supported frequency range and the admissible power caps, and presented in the efficiency–performance plane, one curve per benchmark and polynomial degree. The quantity of interest is the energy saved at a fixed, small loss in time-to-solution.
- Extension beyond the operator evaluation to the solver and preconditioner phases, which introduce outer iterations and more involved access patterns, may move part of the work back onto the host, and involve communication and therefore the power drawn by the network cards, together with a comparison against the host processors of the same systems.

4 Conclusion

This report has extended the co-design work of D3.2 in three directions.

First, the batched (thread-coarsened) mapping of elements to thread blocks resolves the poor low-order performance identified in the first report. By assigning several elements per thread block instead of one, the register and shared-memory capacity left idle by the 32-block-per-SM limit is recovered, raising the achieved fraction of the roofline limit from 27% to 48% for $p = 1$ on the NVIDIA GH200 and delivering a $2\times$ speedup for the Laplace operator. The same mechanism addresses wavefront underutilization on AMD hardware.

Second, the benchmark suite has been extended beyond the scalar, H^1 -conforming kernels of D3.2 to the $H(\text{div})$ -conforming Raviart–Thomas element family, which is required by mixed formulations of the Navier–Stokes equations. The vector mass, mixed divergence and mixed gradient operators have been implemented and measured on both NVIDIA and AMD accelerators, and initial implementations targeting NVIDIA tensor cores in FP64 and TF32 are available.

Third, operator evaluation has been embedded in a complete conjugate gradient solver, allowing the gap between kernel-level (BK) and problem-level (BP) throughput to be quantified. The remaining difference is attributable to gather–scatter overheads and inter-GPU communication rather than to the operator evaluation itself, which confirms that further gains at the application level must come from the solver and communication layers.

The comparison of programming models has been broadened accordingly. High-level directive-based offloading remains convenient but does not reach machine limits for arithmetically intensive high-order kernels, whereas the tile-based approach represented by the Tiny Tensor Compiler approaches the bandwidth limits of Intel GPUs consistently across polynomial orders. Static kernel specialization was confirmed to be a significant lever, with just-in-time compilation strategies keeping the resulting build-time cost manageable.

The energy-efficiency measurements reported in Section 3 are, at the time of writing, still being extended to the full range of target platforms. A consolidated assessment of energy efficiency across the benchmark suite and the participating architectures will therefore be presented in the final project report.

References

- Noel Chalmers and Tim Warburton. Portable high-order finite element kernels i: Streaming operations. *arXiv preprint arXiv:2009.10917*, 2020.
- Salvatore Cielo, Alexander Pöpl, and Ivan Pribec. SYCL for energy-efficient numerical astrophysics: the case of DPEcho. *arXiv preprint arXiv:2508.14117*, 2025.
- Salvatore Cielo, Elmira Birang, Alexander Pöpl, Sajad Azizi, Plamen Dobrev, Margarita Egelhofer, Ivan Pribec, and Gerald Mathias. Node-level performance and energy characterization of flagship science applications on SuperMUC-NG Phase 2. In *IXPUG Workshop, ISC High Performance 2026*, Hamburg, Germany, June 2026. arXiv:2606.23265.
- Paul Fischer, Misun Min, Thilina Rathnayake, Som Dutta, Tzanio Kolev, Veselin Dobrev, Jean-Sylvain Camier, Martin Kronbichler, Tim Warburton, Kasia Świrzydowicz, and Jed Brown. Scalability of high-performance PDE solvers. *The International Journal of High Performance Computing Applications*, 34(5):562–586, 2020. DOI: [10.1177/1094342020915762](https://doi.org/10.1177/1094342020915762).
- Georg Hager, Martin Mayr, Aditya Ujeniya, Lukas Schröder, and Gerhard Wellein. Understanding performance and energy efficiency trade-offs under power capping and frequency scaling scenarios. Keynote, mini-symposium MS045A “Energy-Aware High-Performance Computing I”, WCCM-ECCOMAS 2026, Munich, Germany, 2026. URL <https://wccm-eccomas2026.org/event/contribution/a30ad571-0366-11f1-919d-000c29ddfc0c>. Presented 23 July 2026.
- Martin Kronbichler and Katharina Kormann. Fast matrix-free evaluation of discontinuous Galerkin finite element operators. *ACM Transactions on Mathematical Software*, 45(3):29:1–29:40, 2019. DOI: [10.1145/3325864](https://doi.org/10.1145/3325864).
- NVIDIA Corporation. NVIDIA hopper tuning guide, 2026a. URL <https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html>. Accessed 2026-06-22.
- NVIDIA Corporation. NVIDIA MMA instruction, 2026b. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/#warp-level-matrix-instructions-for-mma>. Accessed 2026-06-23.
- Ivan Pribec, Enes Soydan, Momme Allalen, Gerald Mathias, Ivan Prusak, and Martin Kronbichler. Energy usage of matrix-free finite element methods on modern GPU architectures. Mini-symposium MS045B “Energy-Aware High-Performance Computing II”, WCCM-ECCOMAS 2026, Munich,

Germany, 2026. URL <https://wccm-eccomas2026.org/event/contribution/f0e0ab35-05d3-11f1-919d-000c29ddfc0c>. Presented 23 July 2026.

Kasia Świrydowicz, Noel Chalmers, Ali Karakus, and Tim Warburton. Acceleration of tensor-product operations for high-order finite element methods. *The International Journal of High Performance Computing Applications*, 33(4):735–757, 2019. DOI: [10.1177/1094342018816368](https://doi.org/10.1177/1094342018816368).

Christian R. Trott, Damien Lebrun-Grandié, Daniel Arndt, Jan Ciesko, Vinh Dang, Nathan Ellingwood, Rahulkumar Gayatri, Evan Harvey, Daisy S. Hollman, Dan Ibanez, Nevin Liber, Jonathan Madsen, Jeff Miles, David Poliakoff, Amy Powell, Sivasankaran Rajamanickam, Mikael Simberg, Dan Sunderland, Bruno Turcksin, and Jeremiah Wilke. Kokkos 3: Programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):805–817, 2022. DOI: [10.1109/TPDS.2021.3097283](https://doi.org/10.1109/TPDS.2021.3097283).